Here is a GIFT: Enforcing User Data Isolation in LLM Serving via GPU Information Flow Tracking

Jiacheng Shi[1], Xunjie Wang[1], Cheng Tan[2], Jinyu Gu[1]

[1]Institute of Parallel and Distributed Systems, School of Computer Science, Shanghai Jiao Tong University
[2]Northeastern University

Abstract

LLM serving frameworks process large volumes of user data—often containing sensitive information—on shared infrastructure. Ensuring isolation between users who share the same serving framework (on CPUs) and LLM operators (on GPUs) is critical for privacy protection.

This paper presents GIFT, a GPU Information Flow Tracking system that enforces user data isolation in LLM serving with minimal overhead. Moreover, the design of GIFT is non-intrusive and allows CPU-side serving frameworks to evolve freely. It rests on two key insights. First, *encryption-as-isolation* leverages the observation that CPU components only orchestrate data flow, not content manipulation; thus, per-user encryption can provide isolation without modifying serving logic. Second, GPU kernels exhibit limited and predictable information flows, enabling *static flow analysis*. GIFT precomputes information flow rules for each kernel and uses *decoupled flow tracking*, avoiding instrumentation or GPU stalls.

Furthermore, we extend GIFT to GIFT-CC, which integrates confidential computing to protect against untrusted operating systems and hypervisors (LLM service providers). Implemented on vLLM and DistServe, GIFT and GIFT-CC enforce user data isolation with a 4∼10.7% throughput overhead while maintaining the same latency level.

1 Introduction

Large Language Models (LLMs) have become increasingly integrated into users' daily activities. For example, they assist with drafting emails, answering questions, generating code, and powering personal assistants across both consumer and enterprise applications [7, 15, 17, 3, 40, 41]. As these models handle sensitive tasks, they increasingly observe and process private user data—ranging from personal schedules and medical inquiries to proprietary business documents.

This growing need to handle sensitive data raises fundamental concerns about data leakage and information exposure control. For example, OpenAI temporarily took ChatGPT offline in 2023 due to a caching system bug that allowed some users to see chat titles and histories from other active users [24]. More recently, in 2025, people found that ChatGPT content had appeared in Google search results [8]. These issues are not surprising given the rapid evolution of LLM serving systems—bugs and vulnerabilities [11, 9, 12, 34, 31, 35, 4, 27, 23, 37] are inevitable, and they may lead to unintended data disclosure.

Indeed, an attacker can exploit vulnerabilities in LLM serving systems (e.g., vLLM)—such as CVE-2025-24357 [9]—to perform remote code execution and leak sensitive user data, for example, the contents in the KV cache. This new family of attacks (details in §2.2) enables the attacker to access other users' private conversations without triggering traditional security mechanisms, as the data theft occurs *entirely within GPUs*. These incidents expose a critical question: how can serving frameworks reliably enforce user data isolation while evolving rapidly?

The canonical approach to prevent information from propagating from a source to unintended outputs is *Information Flow Tracking* (IFT), also known as taint tracking [91, 60, 55, 56, 69, 94, 108, 118]. However, applying traditional IFT to LLM serving introduces two challenges:

1) *Complex and fast-evolving LLM systems.* Today's LLM serving frameworks are complex, implemented across multiple languages—typically a mix of Python, CUDA, and C++ or Rust. Moreover, they evolve rapidly, with new features and techniques emerging frequently. For example, vLLM [1] and SGLang [116] have 7.5K and 5K commits in the last year. Traditional IFT is difficult to implement in such sophisticated systems and struggles to remain compatible with rapidly evolving codebases.
2) *High runtime overhead.* Conventional GPU IFT incurs significant overhead [68], which is incompatible with the high-throughput, low-latency requirements of production LLM serving. Ensuring efficient IFT without degrading model serving performance is a major obstacle.

In this paper, we present *GIFT* (*GPU Information Flow Tracking*), a new IFT design that tracks the flow of user data throughout LLM serving. GIFT is the first IFT system for LLM serving that achieves strong isolation guarantees—(a) enforcing user data isolation, (b) preserving the flexibility to evolve the serving framework, and (c) achieving this with minimal performance impact.

GIFT builds on two key observations. First, in LLM serving, the CPU does not manipulate the data directly—instead, the GPU handles all data processing (data-plane)—while the CPU controls the computation by issuing GPU kernels

(control-plane), as shown in Figure 1. This separation means the CPU can operate on *opaque data*, focusing only on placing the right data at the correct GPU memory locations. Second, for serving workloads, GPU kernels are predefined and contain few branches. As a result, their information flows can be analyzed ahead of time and tracked at runtime efficiently.

Based on the first observation, GIFT adopts *encryption-as-isolation*. In LLM serving, the CPU side executes complex and rapidly evolving codebases—such as PyTorch (2.4M lines of code) and Transformers (615K)—that are difficult to secure and often contain many bugs and vulnerabilities; for example, PyTorch has 18 reported CVEs and a number of vulnerabilities are identified in popular serving frameworks [88]. When compromised, an attacker could easily extract user data from CPU memory. Hardening such frameworks is challenging due to their size and evolution rate. GIFT instead encrypts all user data with per-user keys and stores only ciphertext in CPU memory. A small, trusted component decrypts the data immediately before transferring it to the GPU. This approach decouples security from complex frameworks, allowing them to evolve while guaranteeing that any CPU-side data exposure reveals only ciphertext.

Based on the second observation, GIFT introduces a new IFT design that combines *static flow analysis* and *decoupled flow tracking*. Traditional IFT mechanism instruments programs to track data propagation, but this is prohibitively expensive in LLM serving, where GPU kernels are highly optimized and performance-critical. Instead, GIFT analyzes each GPU kernel offline to identify its possible information flows and derive corresponding *IFT rules*—deterministic logic dictating how the flow tracking metadata should be updated for that kernel. During runtime, GIFT executes these rules on the CPU in a decoupled manner and then detects unauthorized data flows without instrumenting the executing kernel or incurring instruction-level overhead on GPU.

GIFT's design is general and applicable to various serving frameworks. To demonstrate this, we implement GIFT on two popular LLM serving systems: vLLM [1] and DistServe [117]. Furthermore, we extend GIFT to operate under a stronger threat model where the underlying system software, including the OS and hypervisor, is untrusted. This extension, called GIFT-CC, leverages hardware confidential computing and is also implemented on both vLLM and DistServe, which can isolate user data from the service providers.

Our evaluation across multiple LLM models shows that GIFT adds negligible latency overhead and incurs at most a 5% throughput reduction under fully saturated serving workloads. GIFT-CC, which integrates confidential computing hardware, introduces up to a 7% throughput overhead compared to non-confidential baselines. In experiments with speculative decoding, GIFT-CC experiences up to a 10.7% throughput drop, largely due to the additional CPU–GPU communication overhead introduced by NVIDIA's confidential computing interface.

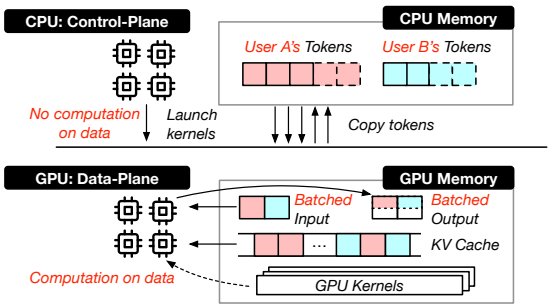

Figure 1. Control/data plane separation in serving systems.

In summary, we make the following contributions: 1) GIFT: The first user data flow tracking approach designed for LLM serving systems; 2) GIFT-CC: An extension of GIFT that leverages confidential computing for a stronger threat model; 3) Real implementation and extensive evaluation demonstrate negligible performance overhead.

2 Motivation, Challenges, and Threat Models

2.1 LLM Serving Systems

Data residency. LLM serving systems, such as vLLM [80] and DistServe [117], orchestrate interactions between CPUs and GPUs to deliver model inference. In these systems, CPUs primarily handle *control-plane* operations such as scheduling, batching, and issuing GPU kernels. The GPUs then execute these kernels to perform computation-intensive tasks (*data-plane* operations) like attention and matrix multiplication. Importantly, CPUs typically do not modify or inspect the content of user data; they only coordinate the execution flow. As a result, most private or user-specific data—including intermediate activations and KV caches used for context reuse—reside on GPUs during serving. Figure 1 overviews this separation.

System evolution and complexity. With the rapid growth of AI, LLM serving systems have evolved—and continue to evolve—into complex infrastructures incorporating numerous optimizations [80, 112, 93, 117, 46, 83, 81, 106]. This complexity arises for good reasons—improving GPU utilization [93, 117], optimizing throughput and latency [46], and enabling dynamic batching [112], caching [80], and speculative decoding [83]. However, these same optimizations introduce more sophisticated interactions between components, making reasoning about data privacy increasingly difficult.

2.2 Data Privacy Threats in LLM Serving Systems

Data privacy is fundamental to LLM services (e.g., ChatGPT, Google Gemini, Claude Code), as user prompts often contain sensitive information [111, 109]. Prior cloud incidents [43, 45, 44, 42] have shown that privacy leaks can arise from diverse sources—bugs, vulnerabilities, misconfigurations, operator errors, and attacks. Moreover, the risk increases as LLM serving frameworks evolve rapidly, incor-

porating larger and increasingly complex codebases.

We conduct a survey of five popular AI infrastructure systems—PyTorch [30], Transformers [18], ONNX [26], TensorFlow [36], and Triton Server [38]—examining their codebase sizes, reported vulnerabilities (i.e., CVEs), and proof-of-concept exploits. Table 1 summarizes the results. Each framework or system typically has a large codebase and exploitable vulnerabilities.

Table 1: Code sizes, CVEs, and proof-of-concept exploits

Component	Lines of Code	#CVE	Proof-of-Concept
vLLM	146K	31	[10], [32], [39]
PyTorch	2.4M	18	[34], [31], [6]
Transformers	615K	4	[23], [37]
ONNX	128K	4	[5], [16]
TensorFlow	1.9M	439	[35], [4], [14]
Triton Server	78K	7	[27]

Beyond theoretical risks, privacy leaks have occurred in practice. In 2023, ChatGPT experienced a major cross-user privacy leak caused by a bug in its serving system [24]. During the incident, some users reported seeing chat histories of others, resulting in OpenAI taking ChatGPT offline temporarily to apply a fix.

GPU confused-deputy attacks. Unlike traditional attacks, those targeting LLM serving systems can exploit GPU-level leakage channels that existing techniques are not able to handle. To illustrate their implications, we conduct an example attack— the compromised serving framework directs the GPU to execute benign kernels with maliciously crafted parameters, which is similar to the confused-deputy attacks [67].

Specifically, vLLM adopts the paged attention [80] mechanism, using a `SelfAttnBlockSpaceManager` object to manage the allocation of KV blocks among different requests. The object provides a method `get_block_table` that returns the KV blocks allocated for a certain request ID. We exploit one remote-code-execution (RCE) vulnerability CVE-2025-24357 [9] in vLLM (before v0.8.0) and thus replace `get_block_table` with a malicious one that always returns the KV blocks of the first request. Thus, the K/V vectors of the first request will always be used to calculate the attention scores of the other requests, leaking one user's data into the responses of other users.

We verify this attack by emulating two users, Alice and Bob, who send prompts "Hello, I am Alice/Bob." to the vLLM server simultaneously. Both users receive the same response: "Hello, Alice! . . . ", meaning that Alice's name is leaked to Bob.

User data isolation. User data isolation is a fundamental (yet often implicit) assumption in LLM serving systems. Users expect that their interactions—queries, prompts, and responses—remain private and separated from those of others. This expectation follows naturally from how models are served: each user session is independent, and there is no legitimate need for data belonging to different users to be mixed or shared. In practice, however, ensuring user data isolation is nontrivial as discussed above.

2.3 Ensuring User Data Isolation via IFT

Information Flow Tracking (IFT) [91, 60, 55, 56, 69, 94, 108, 118] provides a principled mechanism to enforce isolation by monitoring how data propagates within a system. However, applying classic IFT directly to LLM serving systems fails due to two fundamental challenges:

1) On the CPU side, the serving frameworks are complex systems with multiple languages and many third-party libraries. For example, vLLM uses C++ and Python, and imports 244 dynamic libraries. The frameworks also evolve rapidly: vLLM has 7.5K commits in the last year. Implementing IFT logic for each framework version requires high engineering effort. Moreover, there are no mature open-source multi-language dynamic IFT tools, and research work [79] shows instrumentation can cause up to 2.6× overhead on Python programs.
2) On the GPU side, traditional instruction-level instrumentation introduces high runtime overhead. Modern LLM serving is extremely sensitive to latency, so practitioners will not adopt GPU kernels that run 3× slower [68], regardless of the isolation guarantees they provide.

A practical IFT approach for LLM serving must therefore satisfy two criteria: (a) it should integrate easily with existing serving systems without extensive re-engineering, and (b) it should impose minimal performance overhead.

GIFT is *the first* system satisfying these criteria. It provides user data isolation with negligible performance overhead: no observable latency increase and only a 4% throughput reduction. GIFT can adapt to diverse frameworks and hardware configurations; to demonstrate this, we implement GIFT on two serving frameworks: vLLM and DistServe. Orthogonal to the choice of framework, we also build two variants—with and without confidential computing (NVIDIA-CC), a hardware-based security feature [25]. In the following, we use GIFT-CC to denote the versions (for both vLLM and DistServe) with confidential computing enabled, and GIFT to denote those without.

2.4 Threat Models

Threat model for GIFT. GIFT targets a setting where external attackers and buggy code represent the primary threats. GIFT trusts the underlying operating system and hypervisor, including their enforcement of process and VM isolation. In contrast, the LLM serving framework is treated as untrusted—it may be compromised or exhibit arbitrarily malicious behavior including redirecting one user's computation to access another's data (as demonstrated by our example attack, §2.2). To enforce isolation guarantees, GIFT relies on a trusted component, *GIFT monitor* (described in §3), which runs as a standalone VM with 5.6K lines of code

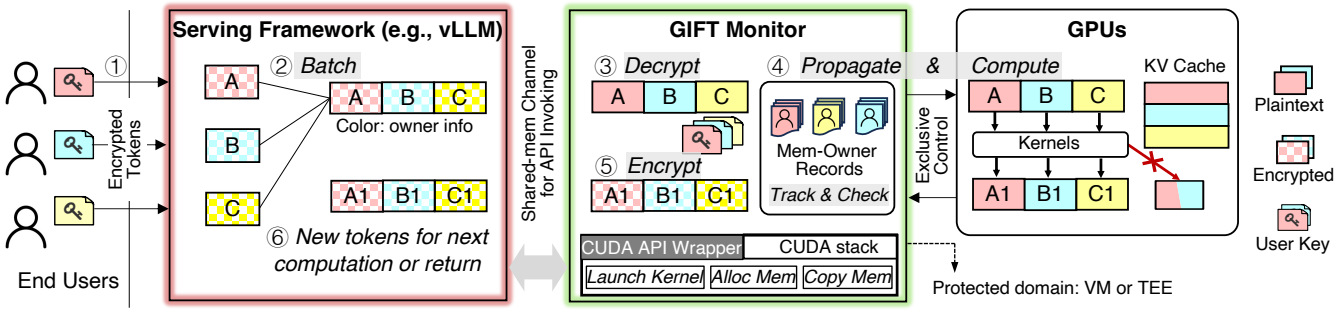

Figure 2. System overview: workflow and architecture.

implemented in Rust.

Threat model for GIFT-CC. GIFT-CC builds on the same core threat assumptions as GIFT: it considers LLM serving frameworks to be exploitable, while trusting the GIFT monitor for enforcement. However, unlike GIFT, GIFT-CC does not trust the operating system or hypervisor. Instead, it leverages confidential computing to ensure the confidentiality of user data even in the presence of a compromised host. In this model, adversaries—such as a privileged administrator of an outsourced LLM service in the cloud (collecting user data for training)—may attempt to inspect CPU memory, issue DMA operations to read GPU memory, or conduct physical attacks like cold booting [65] to break memory confidentiality.

Out-of-scope attacks for GIFT and GIFT-CC. We do not consider side-channel attacks in GIFT or GIFT-CC, as such attacks are orthogonal concerns that can be mitigated using complementary defenses [29]. Similarly, we exclude denial-of-service (DoS) attacks and rely on dedicated DoS detection and mitigation systems to address them. Finally, we do not address issues related to the integrity of inference results, assuming that users can detect significantly degraded or incorrect outputs.

3 GIFT Overview

In GIFT, each user has ownership of their data, including input prompts and all derived data generated during serving, such as KV caches, intermediate tokens, and output data. Each user's private data is encrypted with a user-specific symmetric encryption key, accessible only to the user and the trusted GIFT monitor.

GIFT performs comprehensive ownership tracking[1] throughout the entire data lifecycle, maintaining strict isolation boundaries between users.

System architecture. As shown in Figure 2, GIFT deploys a trusted component (GIFT monitor) within a protected domain to exclusively operate the GPUs, while the untrusted LLM serving framework (e.g., vLLM) runs outside the domain. When the operating system and hypervisor are trusted, GIFT relies on them to allocate GPUs exclusively to the GIFT monitor, and the serving framework and GIFT monitor can be isolated in two VMs. In contrast, when they are untrusted, GIFT monitor can also be protected with a trusted execution environment (TEE) to implement GIFT-CC, which will be introduced in §5.

System workflow. At a high level, GIFT operates as follows.

⓪ During system initialization, GIFT monitor registers all GPU kernels required for executing the target models. In the later serving phase, it acts as the kernel launcher, permitting only registered kernels and rejecting any unauthorized ones. It also loads model parameters into GPU memory and marks them as "parameter"-owned. Besides, it also stores user-specific keys into its protected memory.

① Tokenization of plaintext prompts, which is a lightweight computation, occurs on end users' devices to maintain privacy. The tokenized prompts are encrypted with per-user keys before being sent to the LLM serving framework in the cloud.

② The serving framework (e.g., vLLM) operates entirely on encrypted data while handling control-plane tasks such as continuous batching and kernel scheduling. When launching GPU kernels, it sends a request to GIFT monitor that includes ownership metadata for the batched encrypted inputs.

③ GIFT monitor decrypts user data using the corresponding owners' keys. Crucially, even if the serving framework provides manipulated ownership metadata, the result is only a decryption failure, not a privacy violation.

④ After decryption, GIFT monitor launches the designated GPU kernels, transfers the plaintext data to GPU memory, and records the ownership of GPU memory regions for the kernel inputs. For GPU computation, it tracks and updates GPU memory ownership to serve two functions:

- GIFT monitor needs to enforce strict user data isolation by preventing the framework from mixing data across users (e.g., adding one user's data to another's result) or conducting confused-deputy attacks (§2.2). This mechanism preserves a fundamental invariant in LLM serving: throughout batched execution, each memory byte containing private information must have a singu-

[1] Ownership is equivalent to labels in classic IFT. We use the term ownership to emphasize that GIFT enforces user data isolation rather than the traditional separation between sensitive and non-sensitive data.

lar ownership.

- Before returning data back to the untrusted framework (⑥), GIFT monitor needs to ensure that all data are encrypted using their corresponding owners' keys (⑤). Thereby, the framework always only sees encrypted data, whereas all the plaintext data only resides within the protected domain including the GPUs.

Encryption-as-Isolation. GIFT adopts *encryption-as-isolation* as a core design principle to achieve user data isolation while remaining compatible with the rapid evolution of LLM serving frameworks. The key insight is that encryption can serve as a mechanism for isolation: even if a buggy or malicious framework manipulates ciphertexts, it cannot extract meaningful information or mix data across users, since each user's data is encrypted with a distinct key.

This approach builds on a general system idea—using encryption boundaries to decouple different information flows[2], typically between trusted and untrusted components [47, 101, 86]. Fundamentally, encryption-as-isolation works for GIFT because the CPU-side framework does not modify the semantic content of data; it only orchestrates computation and data movement. From the framework's perspective, ciphertexts and plaintexts are indistinguishable, enabling continuous framework evolution without compromising privacy or requiring changes to GIFT's logic.

A major challenge: IFT overhead. A key challenge in GIFT lies in how ownership is propagated and checked during GPU computation. We have so far described the conceptual workflow without detailing this step, as efficient ownership tracking remains one of the most difficult problems in achieving practical IFT for LLM serving.

Applying traditional IFT techniques directly to GPUs—where each byte or cache line is labeled with ownership information and every instruction is instrumented to propagate these labels—would introduce prohibitive overhead. Such fine-grained tracking is fundamentally incompatible with the performance demands of modern LLM serving. In the following section, we present our insights and solutions to enable efficient GPU memory ownership tracking with near-zero overhead.

4 Near-Zero-Overhead GPU IFT

The end-to-end performance overhead of Information Flow Tracking (IFT) depends on three key factors: (a) the volume of data elements being tracked, (b) the computational complexity of ownership tracking methods, and (c) the runtime effects on the original GPU computation. For LLM serving, such overhead must be near zero—otherwise, no practitioner would adopt it in latency-critical production systems.

[2]IFT systems often require declassification rules to determine which data should be released to lower security levels and how [96]. In GIFT, we use encryption to allow the flow of sensitive data.

To meet this requirement, we systematically analyze the GPU kernels used in LLM serving (§4.1) and develop a new IFT design (§4.2–4.4) that leverages data flow patterns of today's LLM GPU kernels to achieve near-zero-overhead GPU IFT. We begin by presenting our key insights.

4.1 Observations and Insights

We study the three factors of IFT overhead—data volume, tracking method complexity, and runtime effects on GPU—and design corresponding techniques to reduce each cost for LLM serving on GPUs.

Insight 1: Ownership Segmentation. We observe that during LLM serving, data elements belonging to the same owner typically cluster into contiguous GPU memory segments. This spatial locality enables us to implement ownership recording at the segment level rather than the byte or element level, dramatically reducing metadata overhead and complexity.

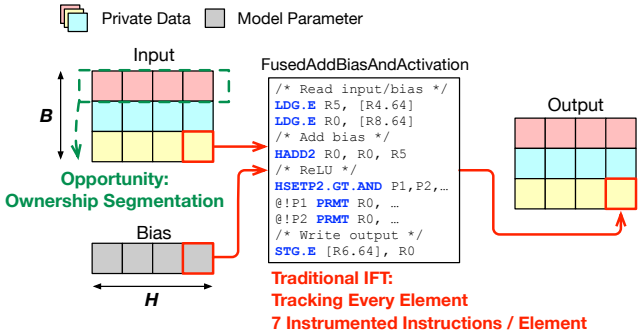

Figure 3. A kernel example and the opportunity for ownership segmentation. B and H indicate batch size and hidden size.

Figure 3 illustrates this principle with a representative feed-forward network kernel that adds a bias vector to each row of an input matrix, computes ReLU, and stores the results in an output matrix. While an element-level method would require maintaining ownership records for each individual element—amounting to $B \times H$ discrete ownership records—our batched approach requires only B records for the entire input or output matrix.

Insight 2: Static Flow Analysis. Traditional IFT techniques [91, 60] are designed for CPU programs that exhibit dynamic semantics with many branches, significant nondeterminism from various sources, and often lack source code access. In contrast, LLM GPU kernels are typically deterministic, have limited control-flow branching, and are available in source form. These properties enable static reasoning and white-box analysis ahead of execution. So, instead of instrumenting binary instructions, we analyze kernel behavior offline and generate IFT rules that define ownership propagation based on kernel arguments and their corresponding control-flow branches, which are typically few in number.

For the kernel example in Figure 3, conventional instrumentation would necessitate instrumenting 7 instructions for propagating ownership. Owing to static analysis, GIFT condenses this to just three streamlined operations required during runtime: (1) verifying the bias vector as model parameters, (2) reading the input row ownership records, and (3) propagating ownership to corresponding output rows. This optimization reduces the computational complexity from $7 \times B \times H$ operations to merely $2 \times B + 1$.

Insight 3: Decoupled Flow Tracking. By leveraging static flow analysis, the ownership propagation is no longer intertwined with the GPU kernel execution. This enables us to decouple ownership tracking and offload it to CPUs (also owing to the reduced computational complexity), effectively hiding the IFT overhead behind GPU computation.

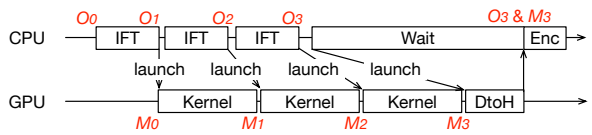

Figure 4. Overlapping IFT on CPU with GPU kernels. O: ownership state, M: GPU memory state, O_i aligns with M_i, DtoH: device-to-host memory copying, Enc: encryption.

Malicious behavior detection during tracking. Before launching a kernel as requested by the LLM serving framework, GIFT monitor performs ownership tracking according to its IFT rule and the provisioned kernel arguments. If there is no unexpected behavior such as ownership mixing, the kernel will be launched; otherwise, GIFT monitor will reject the kernel launch request.

As shown in Figure 4, GIFT monitor always performs ownership tracking before launching kernels and proceeds with subsequent IFT operations without waiting for kernel completion. While this parallelization creates temporary misalignment between the logical ownership state (O) and the actual GPU memory state (M), correctness is maintained for two reasons: First, the ownership state seen by an IFT operation (e.g., O_1) always aligns with the memory state seen by the corresponding kernel (e.g., M_1), because CPU IFT and GPU kernels are both executed sequentially. Second, when data is exported from GPU, the memory state (M_3) always aligns with the ownership state (O_3) used for key selection, because we explicitly wait for the device-to-host memory copying kernels.

Next, we introduce each of the three techniques in detail.

4.2 Ownership Segmentation

GIFT monitor maintains ownership records for each GPU. These records—stored within the GIFT monitor protected domain—indicate readable GPU memory regions and their corresponding data owners.

Recording granularity. Each GPU memory block allocated via `cudaMalloc` contains an n-dimensional tensor that may store data from multiple users. Data belonging to the same user typically forms segments in these tensors, enabling coarse-grained ownership recording.

Figure 3 already shows a concrete example. As another typical example, PagedAttention [80] partitions a 5-dimensional tensor C into KV blocks for different users. Each block consists of multiple K/V vectors, allowing ownership recording at KV block level through tensor slices (e.g., C[1,2,:,0:8,:]).

Ownership data structure. As illustrated in Figure 5, for storing ownership, GIFT uses two-level indexing: the first-level index maps base addresses to tensor records, while the second-level index maps subscript values to ownership records within each tensor.

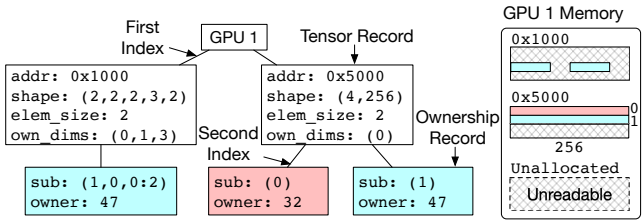

Figure 5. The data structure of ownership records, addr: base address, dtype: data type, odim: ownership-related dimensions, sub: subscript values.

A tensor record consists of four fields: the base address, tensor shape, element size, and ownership-related dimensions. The last field specifies which dimensions determine data ownership, such as (0) for row number in a matrix (e.g., a hidden state matrix) or (0,1,3) for a 5-dimensional tensor (e.g., KV cache blocks).

Each ownership record contains two fields: (1) the subscript values of the ownership-related dimensions, and (2) the owner's user ID, or a special ID for model parameters.

Unowned memory. To prevent leakage of legacy private data, GIFT monitor prohibits reads to any GPU memory region not covered by their ownership records, and clears all ownership records of a tensor when releasing it.

4.3 Static Flow Analysis

The generation of *IFT rules*—program routines that describe how ownership information propagates from kernel inputs to outputs—includes three steps: (1) extracting each kernel's *Information Flow Graph (IFG)*, which captures data dependencies and transformation paths; (2) validating the IFGs against the kernel implementations via symbolic execution [51] to ensure consistency; and (3) generating IFT rules based on the validated IFGs to define expected ownership propagation behavior.

Building IFGs. Figure 6 illustrates the IFGs for four example kernels, where each node represents a GPU memory segment owned by a single user. `AddResidual` adds a residual matrix to an input matrix. `AddBias` adds a bias vector

to each row of an input matrix. `FindMax` finds the index of the maximum element in each row of an input matrix. `MatrixMultiply` multiplies an input matrix by a weight matrix. These IFGs are manually constructed by examining the information flow of kernels and the ownership layout of input tensors for benign inference. Each node is represented by the tensor name, subscript values, and owner. The owner is a variable associated with the subscript values.

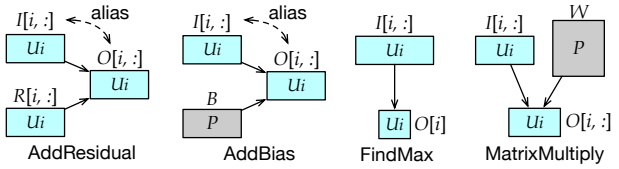

Figure 6. IFGs of four example kernels. I: input, O: output, R: residual, B: bias, W: weight, P: parameter, U_i: owned by user U_i. Alias: one node is an alias of another node.

Validating IFGs. GIFT uses symbolic execution [78] to validate that the manually constructed IFGs accurately reflect the kernel implementation. It first transforms each CUDA kernel into an equivalent C++ version compatible with the widely used symbolic execution engine KLEE [51]. It then verifies that the kernel produces no information flows beyond those specified in its IFG.

Generating IFT rules. Given a validated IFG, the program of its IFT rule should consist of the following steps: (1) identifying tensor records corresponding to accessed tensors by matching their base addresses to kernel arguments; (2) verifying that the kernel arguments are consistent with the expected tensor shapes; (3) checking that tensors occupy distinct memory regions unless explicitly intended to alias, thereby detecting unintended aliasing; (4) ensuring that parameter nodes are correctly labeled as parameter-owned in their ownership records; (5) finally, iterating over all possible subscript values (e.g., all possible i in Figure 6), confirming that non-parameter source nodes share the same owner and assigning this ownership to the corresponding destination nodes.

Semi-automated process and future work. Currently, IFT rule generation in GIFT is semi-automated and requires developer assistance. Although not yet fully automated, this process is a one-time effort per kernel. Moreover, IFT rules are reusable and shareable across models and frameworks. To facilitate this, we have built a library containing rules for 29 standard LLM operators, covering the commonly used models.

Fully automated IFT rule generation is not yet implemented in GIFT due to the lack of a complete symbolic execution engine for CUDA programs. Such an engine would allow GIFT to automatically trace all input values that influence specific outputs. Alternatively, a fully functional transpiler that accurately converts CUDA programs to C++ would enable GIFT to leverage KLEE for this analysis, but no such tool currently exists. At present, GIFT requires human involvement to manually extract IFGs and assist in translating CUDA code to C++ (with semi-automated scripts we build). Reducing this manual effort is our future work.

4.4 Decoupled Flow Tracking

Today's LLM serving frameworks use multiple GPU streams to manage one or more GPUs, for example, a background stream swaps the KV cache while the main stream performs computations. The GIFT monitor creates dedicated CPU threads to handle each GPU stream, and each thread is responsible for ownership propagation and kernel launch. These multi-GPU-stream scenarios require additional concurrency control mechanisms on GPU memory accesses for Decoupled Flow Tracking.

Race conditions. Multi-stream execution introduces potential race conditions that could compromise data privacy. Consider a scenario where an attention kernel in the main stream accesses user A's KV cache while a background stream simultaneously overwrites this cache with user B's data. Although the IFT marks the attention computation result as belonging to user A, the actual result contains user B's private data, constituting a data leak.

To prevent such vulnerabilities, the GIFT monitor must enforce proper synchronization to eliminate both read-write and write-write races on GPU memory. A read-write race occurs when one kernel attempts to read memory that another kernel is actively modifying, while a write-write race involves concurrent writes to the same memory.

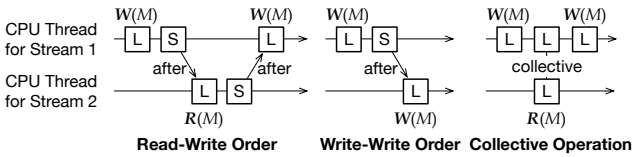

Figure 7. Legal launch order of kernels, L: launch kernel, S: synchronization operation, $R(M)$, $W(M)$: the kernel reads or writes memory segment M.

Concurrency control. As illustrated in Figure 7, the monitor ensures a legal launch order of kernels that access the same memory segment. For this purpose, it needs to record more information.

The ownership record is extended to track the last writer kernel and the last reader kernels for the memory segment. Each kernel is represented by a stream ID and its sequence number within that stream. For each stream, the security monitor also records the last completed kernel. A kernel is deemed completed if it is launched before a synchronization operation, such as cudaMemcpy or cudaStreamSynchronize.

The monitor prevents read-write races by enforcing two invariants. First, when launching a kernel that reads memory segment M, if the last writer of M is on another stream, that writer must have completed. Second, when launching a ker-

nel that writes to memory segment M, if any reader of M is on another stream, that reader must have completed. Write-write races are prevented in a similar manner.

Collective operations. For collective operations like all-reduce in tensor parallelism, multiple GPUs run collaborative kernels that exchange data via communication buffers. In this scenario, there can be both in-flight reader and writer kernels for the buffer. Yet, as shown in Figure 7, the reader on stream 2 (data receiver) does not conflict with the writers on stream 1 as the buffer M is transferred after the first writer completes, and the transfer finishes before the second writer starts executing. The monitor permits this launch order.

4.5 Token Encryption and KV Cache Optimizations

Token encryption. Token encryption is needed when users upload their prompts to the serving framework or GIFT monitor returns the generated tokens to the serving framework. GIFT encrypts each token with a block cipher, instead of encrypting the entire sequence with a stream cipher, because the serving framework needs to perform string operations on the sequence, such as appending newly-generated tokens or splitting prompts into chunks [46].

Simple per-token encryption can expose sequence patterns, as identical tokens yield the same ciphertext with the same user key. GIFT solves this issue with token obfuscation. When encrypting a token, the monitor (or users' devices) extends the 64-bit token to 128 bits by appending a random nonce, then processes it through a block cipher. During decryption, the original token is retrieved by removing the nonce from the 128-bit plaintext. The monitor creates a special ciphertext for the end-of-sequence token to inform the inference framework that the generation is complete.

Secure swap space for KV cache. When GPU memory is insufficient, the serving framework swaps the KV cache to CPU memory. If the KV cache is swapped to the framework's memory, it must be encrypted to prevent privacy leakage. Encrypting and decrypting a large KV cache (e.g., 400 KB per token for OPT-13B) incurs significant overhead.

To avoid this overhead, GIFT monitor allocates a secure swap space in its protected memory for the KV cache, which is inaccessible to the serving framework. It provides the serving framework with interfaces for data transfer between GPU memory and this swap space, and tracks ownership of swapped KV blocks. When blocks are swapped back to GPU memory, their ownership records are restored accordingly.

Caching index tables. Sometimes the IFT rule depends on the *index tables* of the KV cache in GPU memory. For example, GIFT monitor needs to access the block table to determine which KV blocks are accessed by the attention kernel. However, copying index tables from GPU to CPU memory will block IFT, leading to performance overhead. GIFT monitor eliminates this overhead by caching a copy of the index tables in its own memory.

5 GIFT-CC: Isolation with Confidentiality

Motivation. Since LLM serving systems are typically deployed in outsourced environments such as public clouds, users of these services have a legitimate concern about whether the underlying systems (e.g., OSes and hypervisors) and the service providers can be fully trusted. Thereby, we design and implement GIFT-CC to tackle a stronger threat model: how can user data confidentiality be protected when processed in an untrusted LLM serving infrastructure?

TEE Background. Confidential computing has emerged as a promising solution for protecting sensitive data on cloud platforms. CPU vendors provide confidential virtual machines (CVMs) like AMD SEV [2] and Intel TDX [21]; modern GPUs like NVIDIA H100 [25] now support confidential computing as well. GIFT-CC is designed to run LLM serving in a heterogeneous (CPU+GPU) trusted execution environment (TEE), providing user data isolation as well as protecting data confidentiality from the cloud providers.

Difference from GIFT. GIFT-CC adopts the same overall architecture as GIFT, but places its trusted components and GPU computation inside TEEs. Specifically, GIFT monitor is deployed in a CVM that exclusively controls the TEE-enabled GPUs. Before LLM serving, users establish secure communication channels with the monitor using TEE-provided remote attestation [25] to confirm confidential computing is enabled.

The encryption-as-isolation design ensures that all data flowing to untrusted components (i.e., those outside the TEEs) are encrypted. In addition, the CVM communicates with the GPUs through an encrypted PCIe channel, which enforces exclusive GPU control and prevents physical attacks. This encryption incurs additional overhead when copying tokens and KV cache (§4.5) between GIFT monitor and the GPUs.

6 Implementation and Limitations

GIFT's design is general and compatible with existing LLM serving frameworks, allowing them to freely evolve while enforcing user data isolation. To demonstrate this, we implement GIFT on two representative frameworks: vLLM [1] and DistServe [117]—each with two configurations: with and without confidential computing.

We implement GIFT monitor in Rust. It maintains a minimal codebase of 5.6K lines while exposing only 32 interfaces for kernel registration/launching and memory copying/allocation. To intercept CUDA API calls with minimal overhead, we adopt and extend optimized techniques for API remoting [104]. We modify $<$120 lines in each serving framework to track the request owner and call custom APIs.

We implement a library of IFT rules with 29 kernel families[3] for vLLM and DistServe. With this library, GIFT works

[3]Multiple kernels that share a similar information flow, such as elementwise-operation kernels with different functors and data types, are

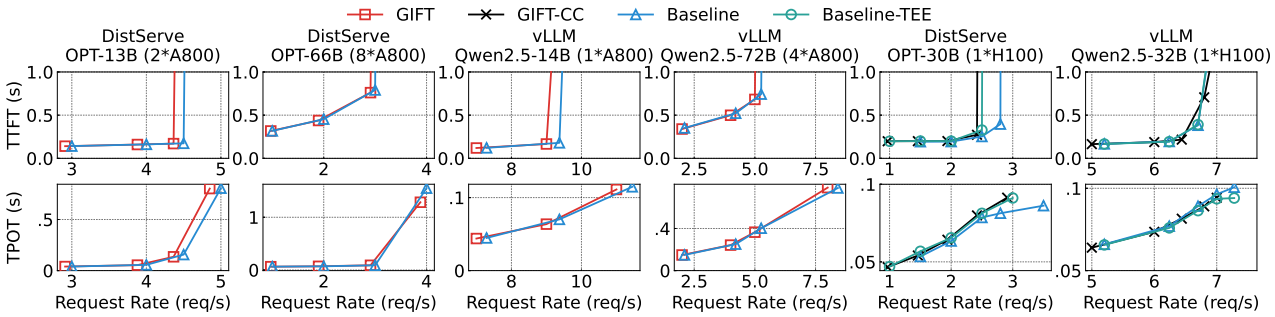

Figure 8. The TTFT and TPOT of ShareGPT benchmark under different request rates on the A800 and H100 machines.

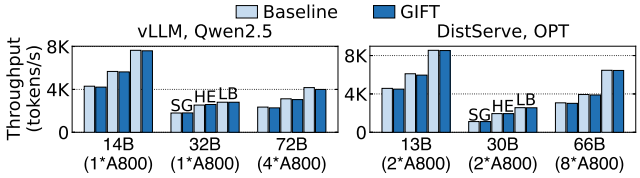

Figure 9. Serving throughput on the A800 machine. SG: ShareGPT, HE: HumanEval, LB: LongBench.

with the models natively supported by these serving frameworks, including Qwen-2.5 [110], Llama-3 [100], Gemma-2 [99], OPT [114], GPT-2 [95], Phi-3 [61].

Limitations. GIFT currently has three main limitations. First, it assumes that the CPU-side framework does not modify user data contents. If this assumption does not hold in some (future) scenarios, GIFT requires changes to the framework to delegate such operations to the GIFT monitor. Second, some GPU kernels may be unavailable for ahead-of-time analysis, either due to technical constraints or legal restrictions. In such cases, users must derive approximate IFT rules through black-box sampling, which may sacrifice soundness. Third, GIFT's current IFT rule generation process is semi-automated, due to the absence of a fully automated CUDA symbolic executor or a comprehensive CUDA transpiler.

7 Performance Evaluation

In this section, we evaluate GIFT's performance and defer its security analysis to the next section. In particular, we mainly answer three questions:

- *End-to-end performance*: How do GIFT and GIFT-CC compare to baselines without user data isolation? (§7.1)
- *Detailed IFT overhead*: What are the costs of tracking information flows for GPU kernels? (§7.2)
- *Optimization impact*: How does each optimization contribute to the overall performance? (§7.3)

Baselines. We compare GIFT against unmodified vLLM [1] and DistServe [117], referred to as *baselines*. For GIFT-CC, we build corresponding TEE-enabled versions that run the original serving frameworks within a TEE with NVIDIA-CC, which we call *baseline-TEE*.

Models. For space saving, we present the experimental results of Qwen2.5 models (14B, 32B, 72B) [110] on vLLM and OPT models (13B, 30B, 66B) [114] on DistServe. The evaluation results are similar for other models.

Framework features. We turn on common optimization features in the serving frameworks. Contiguous batching, paged attention, and pipeline/tensor parallelism are enabled for both vLLM and DistServe experiments. Chunked prefill is supported by vLLM and enabled for its related experiments. Prefill-decoding disaggregation is enabled for DistServe-related experiments.

Benchmarks. ShareGPT [33] (ChatGPT conversations), HumanEval [52] (programming tasks), and LongBench [22] (long text summarization tasks).

Testbed. The evaluation is conducted on two machines: (1) a server with 8 NVIDIA A800 GPUs (NVLINK-connected) and Intel Xeon Platinum CPUs (3.4 GHz), (2) a server with 1 NVIDIA H100 GPU and Intel Xeon Gold CPUs (3.5 GHz).

Baseline-TEE is evaluated only on the H100 platform, as A800 does not support NVIDIA-CC. For GIFT-CC, we modify DistServe to run prefill and decoding stages on the same GPU, as only one H100 GPU is available. For the same reason (insufficient GPUs), OPT-66B and Qwen2.5-72B are not evaluated for GIFT-CC on the H100 machine.

7.1 End-to-End Performance

We experiment GIFT and GIFT-CC with baselines under different models and workloads. The performance metrics are time to first token (TTFT), time per output token (TPOT), and system throughput.

7.1.1 Performance of GIFT (on A800)

Latency. Figure 8 shows the time to first token (TTFT) and time per output token (TPOT) of ShareGPT benchmark on the A800 machine. Compared with the baseline, GIFT incurs up to 5% overhead on the maximum request rate under the same latency SLO (Service Level Objectives). The overhead on other benchmarks is similar.

considered as the same kernel type.

Throughput. Figure 9 presents the serving throughput between GIFT and the baseline on the A800 machine. We use a high request rate to evaluate the maximum throughput of the systems. For the DistServe-based system, both GIFT and the baseline swap the KV cache under this setting, as the GPU memory is insufficient to accommodate all the KV cache required for a decoding batch.

Compared with the baseline, GIFT incurs up to 2.3% and 4.1% overhead for the DistServe-based and the vLLM-based systems, respectively. The overhead is small because the overheads of IFT, encryption, and CUDA API redirection (three main sources that may cause overhead in GIFT) are minimized. First, GIFT hides the IFT overhead with decoupled flow tracking. We analyze kernel execution and IFT latency later (§7.2.1). We also evaluate the effect of optimizations (§7.3). Second, GIFT avoids having the security monitor encrypt the swapped KV cache when the CPU memory is sufficient. We also examine the effect of this optimization in §7.3. The remaining encryption overhead (related to token encryption) is minor due to the small data size. Token encryption accounts for less than 1% of the total serving time for a request. Third, the API redirection overhead in GIFT is less than 0.5%, thanks to the shared-memory-based communication.

7.1.2 Performance of GIFT-CC (on H100)

Latency. We evaluate TTFT and TPOT using ShareGPT benchmark on the H100 platform, as illustrated in Figure 8 (the two columns on the right). Under the same latency SLOs, GIFT-CC exhibits an overhead on maximum request rate of 7% compared to Baseline, and 4% relative to Baseline-TEE. Similar overheads are observed across benchmarks.

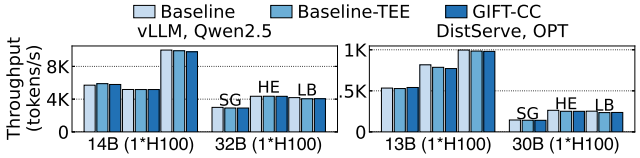

Figure 10. Serving throughput on the H100 machine (TEE enabled), SG: ShareGPT, HE: HumanEval, LB: LongBench.

Throughput. Figure 10 shows the serving throughput of GIFT-CC, Baseline, and Baseline-TEE on the H100 machine. Compared with Baseline, GIFT-CC brings up to 6.7% and 3.2% overhead for the DistServe-based and the vLLM-based systems, respectively. This additional overhead, relative to the A800 results, stems from the encryption and decryption of data transferred between CPU and GPU memory in the H100 TEE environment. The observed overhead is moderate due to the limited intensity of KV cache swapping in our experiment setup. Note that this overhead could be mitigated through emerging TEE hardware architectures [20, 19] and orthogonal optimizations like prefetching [98].

Compared with Baseline-TEE, GIFT-CC brings up to 2.4% and 1.7% overhead for the DistServe-based and the vLLM-based systems, respectively. The overhead is small for the same reasons discussed in §7.1.1.

Experiments with speculative decoding. Figure 11 shows the serving throughput of GIFT-CC, Baseline, and Baseline-TEE with speculative decoding, evaluated on the H100 machine. Compared with Baseline, GIFT-CC incurs up to 10.7% overhead due to the CPU-GPU communication overhead of NVIDIA-CC. Compared with Baseline-TEE, GIFT-CC only incurs up to 2.1% overhead.

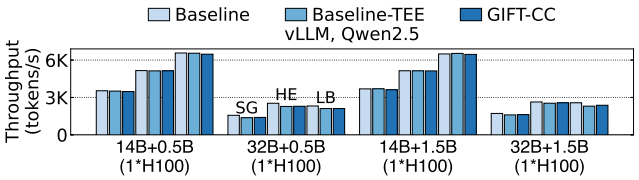

Figure 11. Serving throughput with speculative decoding on the H100 machine (TEE enabled), SG: ShareGPT, HE: HumanEval, LB: LongBench.

7.2 Costs of Information Flow Tracking

In this subsection, we show the costs of IFT for different GPU kernels (§7.2.1), how their overhead varies with model size and input size (§7.2.2), and GIFT's CPU and memory costs (§7.2.3).

7.2.1 GPU Kernel IFT Performance

Figure 12 shows the total execution time and IFT time of each kernel in the DistServe-based system when running the ShareGPT benchmark with OPT-13B on the A800 machine. For every kernel, the IFT time is significantly lower than the kernel execution time (note that the Y-axis is log-scale).

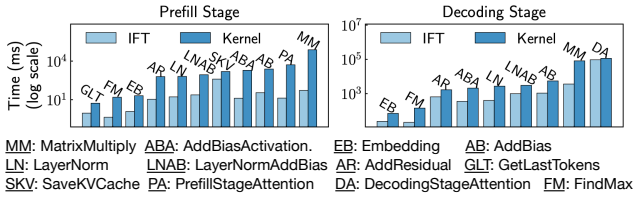

Figure 12. The total execution time and IFT time for each GPU kernel (DistServe, ShareGPT, OPT-13B, A800).

Therefore, the IFT overhead can be hidden with decoupled flow tracking. Specifically, the framework iteratively launches kernels for each model layer, and thus the IFT for one kernel can overlap with the execution of previously-launched kernels. For instance, we observe that the IFT time of a DecodingStageAttention kernel can be hidden within the execution time of a previous MatrixMultiply kernel.

7.2.2 IFT Overhead Analysis

IFT overhead under different model parameter sizes. The parameter size of one kernel depends on the model's hidden size and the number of GPUs used for tensor parallelism.

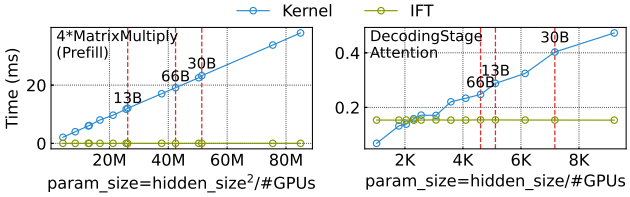

Figure 13. Kernel execution and IFT time under different parameter sizes (A800), #GPUs: tensor parallelism group size.

Figure 13 shows the execution time and IFT time under various parameter sizes for two representative kernels with short or long IFT time. The red lines show the setups used in §7.1.1 (OPT models). As the parameter size increases, the kernel execution time rises, while the IFT time remains constant. The IFT time is constant because the ownership of a hidden state tensor or a KV block can be read or written in a single operation, regardless of their sizes. For the DecodingStageAttention kernel, the IFT time exceeds the execution time when the parameter size is smaller than 2,048, which can be further reduced via multi-threaded IFT (future work).

IFT overhead under different input sizes. The amount of input data involved in kernel execution depends on the batch size (the number of requests) and the context length of each request. Context length refers to the prompt length in prefill stage and the number of previous tokens in decoding stage.

In prefill stage, the kernel execution time grows with larger batch size or context length. Figure 14 illustrates the execution time for MatrixMultiply under different input sizes. The IFT time remains low because it only depends on the batch size, as the hidden states of all tokens in a single prompt share the same owner. In contrast, for most kernels (except for the attention kernel) in decoding stage, the kernel input size is only related to the batch size. Therefore, the gap between the kernel execution time and its IFT time is smaller in decoding stage than in prefill stage (Figure 12).

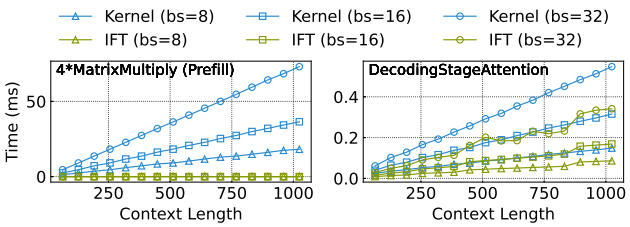

Figure 14. Kernel execution and IFT time under different input sizes (OPT-13B, A800), bs: batch size (#requests).

Among all the kernels, DecodingStageAttention has the longest IFT time. Figure 14 shows that both its execution time and IFT time increase with batch size and context length, because it calculates attention score on each previous token, and its IFT traverses each KV block accessed by the kernel. The IFT time grows slower than the execution time because ownership transfer is much simpler than attention calculation. Across all different input sizes, the IFT time remains lower than the execution time.

7.2.3 CPU and Memory Overhead

Compared with Baseline, GIFT incurs CPU overhead due to IFT, encryption, and API redirection, and needs extra CPU-side memory for recording ownership. Across all setups on both the A800 and H100 machines, the additional CPU usage is below 124% (1.24 cores), and the memory overhead is under 130 MB owing to Ownership Segmentation.

7.3 Effects of Performance Optimizations

In this subsection, we analyze the effects of GIFT techniques for reducing the overhead of IFT and KV cache swapping, based on an evaluation on the DistServe-based system (ShareGPT, OPT-13B, 1×A800).

Fine-grained concurrency control. A naive concurrency control blocks all but one GPU stream when races may occur, which will block the prefill GPU when the decoding GPU fetches KV cache. Our fine-grained concurrency control (§4.4) avoids unnecessary blocking, enabling more concurrency and thus improving GIFT's throughput by 12%.

Secure swap space for KV cache. GIFT monitor saves swapped KV blocks in its protected memory, which avoids the encryption and decryption overhead of copying KV cache between GPU and the serving framework (§4.5). Without this optimization, GIFT experiences an 11% reduction in serving throughput compared with Baseline, while the optimization reduces the overhead to 1.9%.

Caching index tables. GIFT monitor caches index tables used by GPU kernels in the CPU memory (§4.5). In our experiments, there are four kernels, including DecodingStageAttention, that utilize index tables. Without caching, the monitor needs to load the index table from GPU memory when performing IFT for these kernels, which will block until the previously-launched kernels complete and thus expose the IFT overhead on the critical path. The caching optimization increases the throughput of GIFT by 50%.

8 Security Analysis

GIFT ensures user data isolation even when the LLM serving framework is compromised. Furthermore, GIFT-CC protects user data confidentiality against attackers with administrative privileges.

Preventing direct access attacks. If the serving framework (e.g., vLLM) is compromised and an attacker attempts to read user data from CPU memory, the attacker encounters only encrypted content protected by user-specific keys. If the

attacker tries to access plaintext data in either the GIFT monitor memory or GPU memory, such attempts are blocked, by VM isolation in the case of GIFT (which assumes a trusted OS/hypervisor), or by the hardware-enforced TEE boundary in the case of GIFT-CC.

Preventing confused-deputy attacks. If an attacker attempts to launch a malicious GPU kernel that mixes data from different users (e.g., combining the victim's data with the attacker's) in GPU memory, GIFT monitor rejects the kernel according to the generated IFT rules. Even if the attacker issues a sequence of GPU kernels to propagate the victim's data across memory regions before extraction, GIFT monitor continuously tracks memory ownership and ensures that the correct encryption key is applied, preventing the attacker from accessing private user data.

Preventing physical attacks by GIFT-CC. For physical attack vectors, GIFT-CC relies on the security features of NVIDIA-CC hardware. The CPU-side TEE (CVM) employs memory encryption to protect against memory inspection, while the GPU HBM is resistant to common physical attack tools [25]. Additionally, NVIDIA-CC secures PCIe transactions between GPUs and CPUs through TEE-enforced encryption, and extends this protection to NVLINK channels.

TCB Analysis. GIFT-CC's TCB includes TEE hardware and the software in the TEE: a minimal Linux kernel (1M LoC) as the CVM (CPU TEE) OS, CUDA runtime (unknown LoC), CUDA driver (1.4M LoC), and our security monitor (5.6K LoC in Rust). While this may seem substantial, GIFT-CC significantly reduces the overall TCB by excluding the LLM serving framework and its dependencies (3.6M LoC for vLLM) and removes the full-fledged host Linux and other cloud infrastructure (estimated at over 30M LoC) from the TCB.

The remaining TCB, though not minimal, offers strong security for two reasons: (1) it presents a limited attack surface with only 32 exposed interfaces, and (2) it comprises relatively steady and secure components—notably, CUDA has maintained a steady codebase with no public CVEs reported in the past decade [13]. The minimal Linux kernel in the CVM is specifically tailored to remove vulnerable drivers and management interfaces. Moreover, compared to the standard security model of NVIDIA-CC (where CUDA is designed to be in the TCB), GIFT-CC adds only $\sim$5.6K lines of Rust code to the TCB.

Without TEE, GIFT needs to rely on the underlying hypervisor to provide an isolated VM as a protected domain for GIFT monitor. Thereby, the TCB of GIFT further includes the hypervisor. While current GIFT implementation uses vanilla KVM as the hypervisor, existing secure hypervisors [87, 70, 71, 113] could be integrated to reduce the TCB.

9 Related Work

Information flow tracking (IFT). IFT is a general method for tracking data flow at runtime, primarily in CPU programs. It can help detect information leakage, identify abnormal behavior, etc [91, 60, 55, 56, 69, 94, 108, 118, 74]. To reduce the overhead of IFT, prior work [76, 92, 64, 115] has explored offloading flow tracking to a separate CPU thread. While our decoupled flow tracking shares a similar spirit, it is specifically designed for and applied within the GPU context. The initial effort of implementing GPU taint tracking [68] instruments GPU kernels to monitor the data flow in GPU memory. It incurs 2.5–5.7× slowdown on application performance. Compared to prior work, GIFT is the first system to address the unique challenges of implementing IFT for GPU-based LLM serving, achieving near-zero runtime overhead.

GPU kernel analysis. GIFT analyzes GPU kernels to generate IFT rules. Prior work has applied static or dynamic analysis to GPU kernels [54, 84, 49, 50, 77, 82, 85, 72, 89], but none addresses ownership tracking for GPU IFT. For example, POS [72] and Honeycomb [89] analyze memory accesses in GPU kernels for fault tolerance or illegal access prevention, whereas GIFT constructs an information flow graph to enable ownership tracking.

Privacy-protected LLM serving. Cryptography-based solutions such as Homomorphic Encryption [66, 120, 53], Multi-Party Computation [105, 58], Functional Secret Sharing [63], and Differential Privacy [59] can secure user privacy in outsourced LLM serving. However, the significant performance overhead and model modifications that can affect accuracy hinder their adoption. In addition, current studies are primarily evaluated on relatively small models (e.g., 7B).

Protecting AI applications within TEE has been extensively studied [57, 62, 73, 102, 75, 107, 103, 97, 119, 89, 48]. Recent studies on running LLMs inside TEEs [90, 98] show that the performance of TEE-based LLM serving can be close to the non-confidential baseline. GIFT is orthogonal to these systems and focuses on enforcing user data isolation. By combining the two, we implement GIFT-CC, which protects user privacy in a shared LLM inference service.

Apple uses cloud-based LLMs to enhance the user experience of end devices [3]. Apple Private Cloud Compute (PCC) [28] aims to protect privacy for this scenario, employing techniques like OS/libraries tailoring and hardening, and minimizing management interfaces to achieve a narrow attack surface. GIFT-CC shares the same security goal as PCC, but differs in the approach. First, PCC is specific to Apple's proprietary hardware and software, while GIFT-CC is more portable. Second, PCC requires users to trust its entire infrastructure including the inference framework, which necessitates a larger TCB than GIFT-CC.

10 Conclusion

GIFT is the first IFT design for LLM serving systems, which enables user data isolation. Through three key designs, Ownership Segmentation, Static Flow Analysis, and Decoupled Flow Tracking, GIFT introduces minimal performance overhead compared with vanilla LLM serving systems. Furthermore, by integrating confidential computing, GIFT-CC offers stronger privacy protection against untrusted cloud environments.

References

[1] A high-throughput and memory-efficient inference and serving engine for LLMs. https://github.com/vllm-project/vllm.

[2] AMD Secure Encrypted Virtualization (SEV). https://www.amd.com/en/developer/sev.html.

[3] Apple Intelligence. https://www.apple.com/apple-intelligence/.

[4] Arbitrary code execution due to YAML deserialization. https://github.com/advisories/GHSA-r6jx-9g48-2r5r/.

[5] Arbitrary File Overwrite in download_model_with_test_data in onnx/onnx. https://huntr.com/bounties/50235ebd-3410-4ada-b064-1a648e11237e.

[6] Arbitrary File Write via /v1/runs API endpoint in lightning-ai/pytorch-lightning. https://huntr.com/bounties/55a6ac6f-89c7-42ea-86f3-c6e93a2679f3.

[7] ChatGPT. https://chatgpt.com.

[8] ChatGPT users shocked to learn their chats were in Google search results. https://www.reddit.com/r/privacy/comments/1mfb0qz/chatgpt_users_shocked_to_learn_their_chats_were/.

[9] CVE-2025-24357. https://nvd.nist.gov/vuln/detail/CVE-2025-24357.

[10] CVE-2025-24357 Malicious model remote code execution fix bypass with PyTorch $<$ 2.6.0. https://github.com/advisories/GHSA-ggpf-24jw-3fcw.

[11] CVE-2025-32444. https://nvd.nist.gov/vuln/detail/CVE-2025-32444.

[12] CVE-2025-47277. https://nvd.nist.gov/vuln/detail/CVE-2025-47277.

[13] CVEdetails.com. https://www.cvedetails.com.

[14] Data leak in tf.raw_ops.StringNGrams. https://github.com/tensorflow/tensorflow/security/advisories/GHSA-g7p5-5759-qv46.

[15] DeepSeek. https://chat.deepseek.com.

[16] Directory Traversal. https://security.snyk.io/vuln/SNYK-PYTHON-ONNX-2395479.

[17] GitHub Copilot. https://github.com/features/copilot.

[18] huggingface/transformers. https://github.com/huggingface/transformers. GitHub repository.

[19] Integrity and Data Encryption (IDE). https://members.pcisig.com/wg/PCI-SIG/document/16599.

[20] Intel TDX Connect Architecture Specification. https://cdrdv2.intel.com/v1/dl/getContent/773614.

[21] Intel Trust Domain Extensions (Intel TDX). https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html.

[22] LongBench. https://huggingface.co/datasets/THUDM/LongBench.

[23] Malicious model deployed in HF repo to reversed RCE and worm infection by RagRetriever.from_pretrained() with complete bypass of pickle scanning in huggingface/transformers. https://huntr.com/bounties/423611ee-7a2a-442a-babb-3ed2f8385c16.

[24] March 20 ChatGPT outage: Here's what happened. https://openai.com/index/march-20-chatgpt-outage/.

[25] NVIDIA Confidential Computing. https://images.nvidia.cn/aem-dam/en-zz/Solutions/data-center/HCC-Whitepaper-v1.0.pdf.

[26] onnx/onnx. https://github.com/onnx/onnx. GitHub repository.

[27] Preauth RCE on NVIDIA Triton Server. https://sites.google.com/site/zhiniangpeng/blogs/Triton-RCE.

[28] Private Cloud Compute Security Guide. https://security.apple.com/documentation/private-cloud-compute/.

[29] Private Cloud Compute Security Guide / Stateless Inference. https://security.apple.com/documentation/private-cloud-compute/statelessinference#Language-model-inference.

[30] pytorch/pytorch. https://github.com/pytorch/pytorch. GitHub repository.

[31] RCE via Property/Class Pollution due to state change endpoint in lightning-ai/pytorch-lightning. https://huntr.com/bounties/486add92-275e-4a7b-92f9-42d84bc759da.

[32] Remote Code Execution by Pickle Deserialization via MessageQueue.dequeue() Broadcast Communication API in vllm-project/vllm. https://huntr.com/bounties/00136195-11e0-4ad0-98d5-72db066e867f.

[33] ShareGPT. https://sharegpt.com.

[34] Shelltorch Explained: Multiple Vulnerabilities in Pytorch Model Server (Torchserve) Walkthrough. https://www.oligo.security/blog/shelltorch-explained-multiple-vulnerabilities-in-pytorch-model-server.

[35] TensorFlow Python Code Injection: More eval() Woes. https://jfrog.com/blog/tensorflow-python-code-injection-more-eval-woes/.

[36] tensorflow/tensorflow. https://github.com/tensorflow/tensorflow. GitHub repository.

[37] Transformers has a Deserialization of Untrusted Data vulnerability in huggingface/transformers. https://huntr.com/bounties/b3c36992-5264-4d7f-9906-a996efafba8f.

[38] triton-inference-server/server. https://github.com/triton-inference-server/server. GitHub repository.

[39] vLLM Allows Remote Code Execution via PyNcclPipe Communication Service. https://github.com/advisories/GHSA-hjq4-87xh-g4fv.

[40] Voice Assistant Celia - HUAWEI Global. https://consumer.huawei.com/en/emui/celia/.

[41] Welcome to Copilot on Windows. https://support.microsoft.com/en-us/windows/welcome-to-copilot-on-windows-675708af-8c16-4675-afeb-85a5a476ccb0.

[42] Capital One: Facts 2019. https://www.capitalone.com/digital/facts2019/, 2019.

[43] CVE-2021-44228 (Log4Shell). https://nvd.nist.gov/vuln/detail/CVE-2021-44228, 2021.

[44] Reflecting on the 2023 Toyota data breach. https://cloudsecurityalliance.org/blog/2025/07/21/reflecting-on-the-2023-toyota-data-breach, 2025.

[45] Unpacking the 2024 Snowflake data breach. https://cloudsecurityalliance.org/blog/2025/05/07/unpacking-the-2024-snowflake-data-breach, 2025.

[46] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In A. Gavrilovska and D. B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 117–134. USENIX Association, 2024.

[47] A. Arasu, S. Blanas, K. Eguro, R. Kaushik, D. Kossmann, R. Ramamurthy, and R. Venkatesan. Orthogonal security with cipherbase. In *6th Biennial Conference on Innovative Data Systems Research (CIDR'13)*, January 2013.

[48] R. Bahmani, F. Brasser, G. Dessouky, P. Jauernig, M. Klimmek, A. Sadeghi, and E. Stapf. CURE: A security architecture with customizable and resilient enclaves. In M. D. Bailey and R. Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1073–1090. USENIX Association, 2021.

[49] A. Betts, N. Chong, A. F. Donaldson, J. Ketema, S. Qadeer, P. Thomson, and J. Wickerson. The design and implementation of a verification technique for GPU kernels. *ACM Trans. Program. Lang. Syst.*, 37(3):10:1–10:49, 2015.

[50] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson. Gpuverify: a verifier for GPU kernels. In G. T. Leavens and M. B. Dwyer, editors, *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2012, part of SPLASH 2012, Tucson, AZ, USA, October 21-25, 2012*, pages 113–132. ACM, 2012.

[51] C. Cadar, D. Dunbar, and D. R. Engler. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In R. Draves and R. van Renesse, editors, *8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008, December 8-10, 2008, San Diego, California, USA, Proceedings*, pages 209–224. USENIX Association, 2008.

[52] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.

[53] T. Chen, H. Bao, S. Huang, L. Dong, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei. The-x: Privacy-preserving transformer inference with homomorphic encryption, 2022.

[54] W. Chiang, G. Gopalakrishnan, G. Li, and Z. Rakamaric. Formal analysis of GPU programs with atomics via conflict-directed delay-bounding. In G. Brat, N. Rungta, and A. Venet, editors, *NASA Formal Methods, 5th International Symposium, NFM 2013, Moffett Field, CA, USA, May 14-16, 2013. Proceedings*, volume 7871 of *Lecture Notes in Computer Science*, pages 213–228. Springer, 2013.

[55] J. Chow, B. Pfaff, T. Garfinkel, K. Christopher, and M. Rosenblum. Understanding data lifetime via whole system simulation. In M. Blaze, editor, *Proceedings of the 13th USENIX Security Symposium, August 9-13, 2004, San Diego, CA, USA*, pages 321–336. USENIX, 2004.

[56] J. A. Clause, W. Li, and A. Orso. Dytan: a generic dynamic taint analysis framework. In D. S. Rosenblum and S. G. Elbaum, editors, *Proceedings of the ACM/SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2007, London, UK, July 9-12, 2007*, pages 196–206. ACM, 2007.

[57] Y. Deng, C. Wang, S. Yu, S. Liu, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang. Strongbox: A GPU TEE on arm endpoints. In H. Yin, A. Stavrou, C. Cremers, and E. Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 769–783. ACM, 2022.

[58] Y. Dong, W. jie Lu, Y. Zheng, H. Wu, D. Zhao, J. Tan, Z. Huang, C. Hong, T. Wei, and W. Chen. Puma: Secure inference of llama-7b in five minutes, 2023.

[59] M. Du, X. Yue, S. S. M. Chow, T. Wang, C. Huang, and H. Sun. Dp-forward: Fine-tuning and inference on language models with differential privacy in forward pass. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, CCS '23, page 2665–2679, New York, NY, USA, 2023. Association for Computing Machinery.

[60] W. Enck, P. Gilbert, B. Chun, L. P. Cox, J. Jung, P. D. McDaniel, and A. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In R. H. Arpaci-Dusseau and B. Chen, editors, *9th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2010, October 4-6, 2010, Vancouver, BC, Canada, Proceedings*, pages 393–407. USENIX Association, 2010.

[61] M. A. et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.

[62] E. Feng, D. Feng, D. Du, Y. Xia, and H. Chen. snpu: Trusted execution environments on integrated npus. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*, pages 708–723. IEEE, 2024.

[63] K. Gupta, N. Jawalkar, A. Mukherjee, N. Chandran, D. Gupta, A. Panwar, and R. Sharma. Sigma: Secure gpt inference with function secret sharing. *Cryptology ePrint Archive*, 2023.

[64] J. Ha, M. Arnold, S. M. Blackburn, and K. S. McKinley. A concurrent dynamic analysis framework for multicore hardware. In S. Arora and G. T. Leavens, editors, *Proceedings of the 24th Annual ACM SIGPLAN Conference*

on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2009, October 25-29, 2009, Orlando, Florida, USA, pages 155–174. ACM, 2009.

[65] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten. Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM*, 52(5):91–98, May 2009.

[66] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang. Iron: private inference on transformers. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2024. Curran Associates Inc.

[67] N. Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Oper. Syst. Rev.*, 22(4):36–38, 1988.

[68] A. B. Hayes, L. Li, M. Hedayati, J. He, E. Z. Zhang, and K. Shen. GPU taint tracking. In D. D. Silva and B. Ford, editors, *Proceedings of the 2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, pages 209–220. USENIX Association, 2017.

[69] A. Ho, M. A. Fetterman, C. Clark, A. Warfield, and S. Hand. Practical taint-based protection using demand emulation. In Y. Berbers and W. Zwaenepoel, editors, *Proceedings of the 2006 EuroSys Conference, Leuven, Belgium, April 18-21, 2006*, pages 29–41. ACM, 2006.

[70] A. V. Hof and J. Nieh. BlackBox: A container security monitor for protecting containers on untrusted operating systems. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 683–700, Carlsbad, CA, July 2022. USENIX Association.

[71] O. S. Hofmann, S. Kim, A. M. Dunn, M. Z. Lee, and E. Witchel. Inktag: secure applications on an untrusted operating system. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, page 265–278, New York, NY, USA, 2013. Association for Computing Machinery.

[72] Z. Huang, X. Wei, Y. Hao, R. Chen, M. Han, J. Gu, and H. Chen. PARALLELGPUOS: A concurrent os-level GPU checkpoint and restore system using validated speculation. *CoRR*, abs/2405.12079, 2024.

[73] T. Hunt, Z. Jia, V. Miller, A. Szekely, Y. Hu, C. J. Rossbach, and E. Witchel. Telekine: Secure computing with cloud gpus. In R. Bhagwan and G. Porter, editors, *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, pages 817–833. USENIX Association, 2020.

[74] T. Hunt, Z. Zhu, Y. Xu, S. Peter, and E. Witchel. Ryoan: A distributed sandbox for untrusted computation on secret data. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 533–549, Savannah, GA, Nov. 2016. USENIX Association.

[75] I. Jang, A. Tang, T. Kim, S. Sethumadhavan, and J. Huh. Heterogeneous isolated execution for commodity gpus. In I. Bahar, M. Herlihy, E. Witchel, and A. R. Lebeck, editors, *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, pages 455–468. ACM, 2019.

[76] K. Jee, V. P. Kemerlis, A. D. Keromytis, and G. Portokalidis. Shadowreplica: efficient parallelization of dynamic data flow tracking. In A. Sadeghi, V. D. Gligor, and M. Yung, editors, *2013 ACM SIGSAC Conference on Computer and Communications Security, CCS'13, Berlin, Germany, November 4-8, 2013*, pages 235–246. ACM, 2013.

[77] A. K. Kamath and A. Basu. iguard: In-gpu advanced race detection. In R. van Renesse and N. Zeldovich, editors, *SOSP '21: ACM SIGOPS 28th Symposium on Operating Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, pages 49–65. ACM, 2021.

[78] J. C. King. Symbolic execution and program testing. *Commun. ACM*, 19(7):385–394, 1976.

[79] J. Kreindl, D. Bonetta, L. Stadler, D. Leopoldseder, and H. Mössenböck. Multi-language dynamic taint analysis in a polyglot virtual machine. In S. Marr, editor, *MPLR '20: 17th International Conference on Managed Programming Languages and Runtimes, Virtual Event, UK, November 4-6, 2020*, pages 15–29. ACM, 2020.

[80] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, editors, *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023.

[81] W. Lee, J. Lee, J. Seo, and J. Sim. Infinigen: Efficient generative inference of large language models with dynamic KV cache management. In A. Gavrilovska and D. B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 155–172. USENIX Association, 2024.

[82] A. Leung, M. Gupta, Y. Agarwal, R. Gupta, R. Jhala, and S. Lerner. Verifying GPU kernels by test amplification. In J. Vitek, H. Lin, and F. Tip, editors, *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, pages 383–394. ACM, 2012.

[83] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[84] G. Li and G. Gopalakrishnan. Scalable smt-based verification of GPU kernel functions. In G. Roman and A. van der Hoek, editors, *Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2010, Santa Fe, NM, USA, November 7-11, 2010*, pages 187–196. ACM, 2010.

[85] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan. GKLEE: concolic verification and test generation for gpus. In J. Ramanujam and P. Sadayappan, editors, *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2012,*

New Orleans, LA, USA, February 25-29, 2012, pages 215–224. ACM, 2012.

[86] M. Li, X. Zhao, L. Chen, C. Tan, H. Li, S. Wang, Z. Mi, Y. Xia, F. Li, and H. Chen. Encrypted databases made secure yet maintainable. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 117–133, Boston, MA, July 2023. USENIX Association.

[87] S.-W. Li, X. Li, R. Gu, J. Nieh, and J. Zhuang Hui. A secure and formally verified linux kvm hypervisor. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1782–1799, 2021.

[88] T. Liu, Z. Deng, G. Meng, Y. Li, and K. Chen. Demystifying rce vulnerabilities in llm-integrated apps, 2024.

[89] H. Mai, J. Zhao, H. Zheng, Y. Zhao, Z. Liu, M. Gao, C. Wang, H. Cui, X. Feng, and C. Kozyrakis. Honeycomb: Secure and efficient GPU executions via static validation. In R. Geambasu and E. Nightingale, editors, *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2023, Boston, MA, USA, July 10-12, 2023*, pages 155–172. USENIX Association, 2023.

[90] A. Mohan, M. Ye, H. Franke, M. Srivatsa, Z. Liu, and N. M. Gonzalez. Securing ai inference in the cloud: Is cpu-gpu confidential computing ready? In *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*, pages 164–175, 2024.

[91] J. Newsome and D. X. Song. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In *Proceedings of the Network and Distributed System Security Symposium, NDSS 2005, San Diego, California, USA*. The Internet Society, 2005.

[92] E. B. Nightingale, D. Peek, P. M. Chen, and J. Flinn. Parallelizing security checks on commodity hardware. In S. J. Eggers and J. R. Larus, editors, *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2008, Seattle, WA, USA, March 1-5, 2008*, pages 308–318. ACM, 2008.

[93] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*, pages 118–132. IEEE, 2024.

[94] F. Qin, C. Wang, Z. Li, H. Kim, Y. Zhou, and Y. Wu. LIFT: A low-overhead practical information flow tracking system for detecting security attacks. In *39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-39 2006), 9-13 December 2006, Orlando, Florida, USA*, pages 135–148. IEEE Computer Society, 2006.

[95] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019.

[96] A. Sabelfeld and D. Sands. Declassification: Dimensions and principles. *J. Comput. Secur.*, 17(5):517–548, Oct. 2009.

[97] S. Sridhara, A. Bertschi, B. Schlüter, M. Kuhne, F. Aliberti, and S. Shinde. ACAI: extending arm confidential computing architecture protection from cpus to accelerators. *CoRR*, abs/2305.15986, 2023.

[98] Y. Tan, C. Tan, Z. Mi, and H. Chen. Pipellm: Fast and confidential large language model services with speculative pipelined encryption. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 843–857, 2025.

[99] G. Team. Gemma 2: Improving open language models at a practical size, 2024.

[100] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[101] D. Vinayagamurthy, A. Gribov, and S. Gorbunov. Stealthdb: a scalable encrypted database with full SQL query support. *Proc. Priv. Enhancing Technol.*, 2019(3):370–388, 2019.

[102] S. Volos, K. Vaswani, and R. Bruno. Graviton: Trusted execution environments on gpus. In A. C. Arpaci-Dusseau and G. Voelker, editors, *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 681–696. USENIX Association, 2018.

[103] C. Wang, F. Zhang, Y. Deng, K. Leach, J. Cao, Z. Ning, S. Yan, and Z. He. CAGE: complementing arm CCA with GPU extensions. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society, 2024.

[104] T. Wang, Z. Chen, X. Wei, J. Gu, R. Chen, and H. Chen. Characterizing network requirements for GPU API remoting in AI applications. *CoRR*, abs/2401.13354, 2024.

[105] Y. Wang, G. E. Suh, W. Xiong, B. Lefaudeux, B. Knott, M. Annavaram, and H.-H. S. Lee. Characterization of mpc-based private inference for transformer-based models. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 187–197, 2022.

[106] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In E. Witchel, C. J. Rossbach, A. C. Arpaci-Dusseau, and K. Keeton, editors, *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP 2024, Austin, TX, USA, November 4-6, 2024*, pages 640–654. ACM, 2024.

[107] X. Wu, D. J. Tian, and C. H. Kim. Building GPU tees using CPU secure enclaves with gevisor. In *Proceedings of the 2023 ACM Symposium on Cloud Computing, SoCC 2023, Santa Cruz, CA, USA, 30 October 2023 - 1 November 2023*, pages 249–264. ACM, 2023.

[108] W. Xu, S. Bhatkar, and R. Sekar. Taint-enhanced policy enforcement: A practical approach to defeat a wide range of attacks. In A. D. Keromytis, editor, *Proceedings of the 15th USENIX Security Symposium, Vancouver, BC, Canada, July 31 - August 4, 2006*. USENIX Association, 2006.

[109] B. Yan, K. Li, M. Xu, Y. Dong, Y. Zhang, Z. Ren, and X. Cheng. On protecting the data privacy of large language models (llms): A survey, 2024.

[110] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li,

D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. *CoRR*, abs/2412.15115, 2024.

[111] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211, June 2024.

[112] G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun. Orca: A distributed serving system for transformer-based generative models. In M. K. Aguilera and H. Weatherspoon, editors, *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 521–538. USENIX Association, 2022.

[113] F. Zhang, J. Chen, H. Chen, and B. Zang. Cloudvisor: retrofitting protection of virtual machines in multi-tenant cloud with nested virtualization. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, SOSP '11, page 203–216, New York, NY, USA, 2011. Association for Computing Machinery.

[114] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. T. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer. OPT: open pre-trained transformer language models. *CoRR*, abs/2205.01068, 2022.

[115] Q. Zhao, I. Cutcutache, and W. Wong. Pipa: pipelined profiling and analysis on multi-core systems. In M. L. Soffa and E. Duesterwald, editors, *Sixth International Symposium on Code Generation and Optimization (CGO 2008), April 5-9, 2008, Boston, MA, USA*, pages 185–194. ACM, 2008.

[116] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng. Efficiently programming large language models using sglang. *CoRR*, abs/2312.07104, 2023.

[117] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In A. Gavrilovska and D. B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 193–210. USENIX Association, 2024.

[118] D. Y. Zhu, J. Jung, D. Song, T. Kohno, and D. Wetherall. Tainteraser: protecting sensitive data leaks using application-level taint tracking. *ACM SIGOPS Oper. Syst. Rev.*, 45(1):142–154, 2011.

[119] J. Zhu, R. Hou, X. Wang, W. Wang, J. Cao, B. Zhao, Z. Wang, Y. Zhang, J. Ying, L. Zhang, and D. Meng. Enabling rack-scale confidential computing using heterogeneous trusted execution environment. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*, pages 1450–1465. IEEE, 2020.

[120] I. Zimerman, M. Baruch, N. Drucker, G. Ezov, O. Soceanu, and L. Wolf. Converting transformers to polynomial form for secure inference over homomorphic encryption, 2023.